Métodos de Optimización

Clase 1: Introducción y conceptos matemáticos básicos

Whady Felipe Florez, PhD. University of Wales- Wessex Institute of Technology, UK.

27 de octubre de 2026

Universidad POntificia Bolivariana]

Contenido

1. ¿Qué es optimizar?
2. Derivada de una función de una variable
3. Funciones de varias variables y parciales
4. El vector gradiente
5. Curvas de nivel
6. Derivada direccional y máx. crecimiento
7. Monotonía
8. Puntos críticos
9. La matriz Hessiana
10. Serie de Taylor
11. Convexidad
12. Máx./mín. locales vs. globales
13. Optimización sin restricciones
14. Restricciones de igualdad (Lagrange)
15. Restricciones de desigualdad (KKT)
16. Síntesis de la clase
17. Ejercicios propuestos
18. Referencias

¿Qué es optimizar?

¿Qué es optimizar?

Idea clave

Optimizar es **elegir, entre muchas alternativas posibles, la mejor según un criterio numérico**: el menor costo, el mayor beneficio, el menor error, el menor tiempo, la mayor resistencia, etc.

Ejemplos cotidianos de optimización:

- Una empresa decide cuánto producir de cada artículo para **maximizar utilidad** sin exceder su capacidad.
- Un ingeniero diseña una viga que **minimiza el peso** sin sobrepasar el esfuerzo admisible.
- Un algoritmo de aprendizaje automático ajusta parámetros para **minimizar el error** de predicción.
- Un repartidor busca la ruta que **minimiza la distancia** recorrida.
- Un profesor de secundaria que cerca un jardín rectangular con cuerda fija busca el rectángulo de **área máxima**.

Formulación matemática general

Un problema de optimización se escribe, en su forma más general, como

$$\min_{\mathbf{x}\in\Omega\subseteq\mathbb{R}^n} \; f(\mathbf{x})$$

Elementos del problema

- $f : \mathbb{R}^n \to \mathbb{R}$: **función objetivo**.
- $\mathbf{x} = (x_1, \ldots, x_n)$: **variables de decisión**.
- Ω: **conjunto factible** (restricciones).
- Si $\Omega = \mathbb{R}^n$: optimización **sin restricciones**.
- Si Ω se define con ecuaciones/desigualdades: optimización **con restricciones**.

Maximizar = minimizar $-f$

$$\max_{\mathbf{x}\in\Omega} f(\mathbf{x}) \;\equiv\; \min_{\mathbf{x}\in\Omega} \big(-f(\mathbf{x})\big)$$

Por eso casi toda la teoría se presenta en términos de *minimización*, sin pérdida de generalidad.

Objetivo de esta clase

Antes de estudiar *métodos* de optimización (descenso de gradiente, Newton, programación lineal, etc.), necesitamos un lenguaje matemático común. Hoy construimos ese lenguaje:

- Derivadas ordinarias y parciales
- El gradiente
- Curvas de nivel
- Derivada direccional
- Monotonía
- Puntos críticos y Hessiana
- Serie de Taylor
- Convexidad
- Mínimos/máximos locales y globales
- Optimización con restricciones de igualdad
- Optimización con restricciones de desigualdad

Todos los conceptos se ilustran con gráficas y **código Python** reproducible (disponible como scripts independientes).

Derivada de una función de una variable

La derivada: definición

Definición

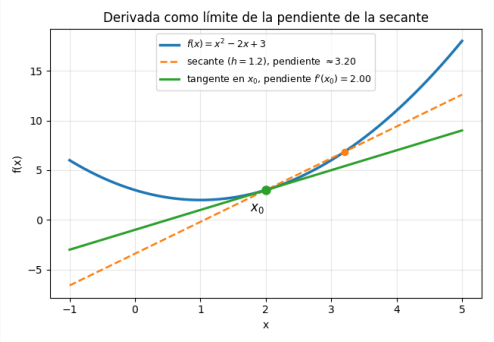
La derivada de $f : \mathbb{R} \to \mathbb{R}$ en x_0 es

$$f'(x_0) = \lim_{h \to 0} \frac{f(x_0 + h) - f(x_0)}{h}$$

cuando ese límite existe.

- El cociente $\dfrac{f(x_0 + h) - f(x_0)}{h}$ es la pendiente de la **recta secante** que une $(x_0, f(x_0))$ con $(x_0 + h, f(x_0 + h))$.
- Al hacer $h \to 0$, la secante se convierte en la **recta tangente**: $f'(x_0)$ es su pendiente.
- Interpretación física: **tasa instantánea de cambio** de f respecto a x.

La derivada: interpretación geométrica



Al reducir h, la recta secante (naranja) converge a la recta tangente (verde) en x_0.

Derivada en Python: numérica y simbólica

Aproximación numérica (diferencias finitas):

```
def f(x):
    return x**2 - 2*x + 3

def derivada_numerica(f, x0, h=1e-5):
    return (f(x0 + h) - f(x0)) / h

print(derivada_numerica(f, 2.0))
# -> 2.00001   (exacto: f'(x)=2x-2 -> 2)
```

Cálculo simbólico (sympy):

```
import sympy as sp

x = sp.symbols('x')
f = x**2 - 2*x + 3
fprime = sp.diff(f, x)
print(fprime)           # 2*x - 2
print(fprime.subs(x, 2))  # 2
```

Script completo: `scripts/01_derivada_tangente.py`

Reglas básicas de derivación (recordatorio)

Función	Derivada
c (constante)	0
x^n	$n\,x^{n-1}$
e^x	e^x
$\ln x$	$1/x$
$\sin x$	$\cos x$
$\cos x$	$-\sin x$

Regla	Fórmula
Suma	$(f+g)' = f' + g'$
Producto	$(fg)' = f'g + fg'$
Cociente	$(f/g)' = \frac{f'g - fg'}{g^2}$
Cadena	$(f \circ g)'(x) = f'(g(x))g'(x)$

Para el público no especializado

Solo necesitamos saber *qué representa* la derivada (pendiente / razón de cambio) y aplicar estas reglas mecánicamente; no se requiere dominar todas las técnicas de cálculo.

Funciones de varias variables y derivadas parciales

Funciones de varias variables

En optimización casi siempre hay **más de una variable de decisión**: cantidades a producir de varios artículos, dimensiones de una pieza, parámetros de un modelo, etc.

$$f : \mathbb{R}^n \to \mathbb{R}, \qquad f(x_1, x_2, \ldots, x_n)$$

Ejemplo

Costo de fabricar una caja rectangular de dimensiones x, y, z:

$$f(x, y, z) = 2(xy + yz + xz) \cdot c$$

con c el costo por unidad de área de material.

Nos enfocaremos en $f(x, y) : \mathbb{R}^2 \to \mathbb{R}$ porque se puede **visualizar** como una superficie; todo generaliza a n variables.

Derivadas parciales: definición

Definición

La derivada parcial de $f(x, y)$ respecto a x en (x_0, y_0) es

$$\frac{\partial f}{\partial x}(x_0, y_0) = \lim_{h \to 0} \frac{f(x_0 + h, y_0) - f(x_0, y_0)}{h}$$

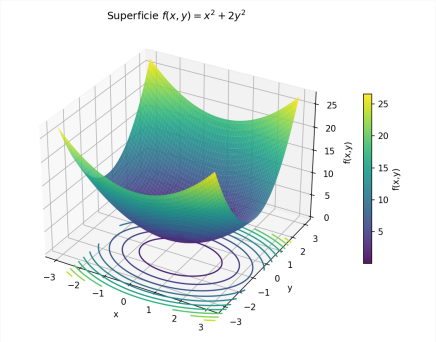
(se deriva respecto a x **dejando** y **fija**, y análogamente para $\partial f / \partial y$).

- Notaciones equivalentes: $\dfrac{\partial f}{\partial x} = f_x = \partial_x f$.
- **Regla práctica:** se derivan igual que en una variable, tratando las demás variables como constantes.

Ejemplo

$$f(x, y) = x^2 + 2y^2 \;\Rightarrow\; \frac{\partial f}{\partial x} = 2x, \quad \frac{\partial f}{\partial y} = 4y$$

Interpretación geométrica de la derivada parcial



$\partial f/\partial x$ en (x_0, y_0): pendiente de la curva que se obtiene al **cortar la superficie con el plano** $y = y_0$.
Análogamente, $\partial f/\partial y$ corresponde al plano $x = x_0$.

Generando superficies en Python

```python
import numpy as np
import matplotlib.pyplot as plt

f = lambda x, y: x**2 + 2*y**2

x = np.linspace(-3, 3, 120)
y = np.linspace(-3, 3, 120)
X, Y = np.meshgrid(x, y)       # malla 2D de puntos
Z = f(X, Y)                     # evaluamos f en toda la malla

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, Y, Z, cmap="viridis")
ax.set_xlabel("x"); ax.set_ylabel("y"); ax.set_zlabel("f(x,y)")
plt.show()
```

Script completo: `scripts/02_superficie_3d.py`

El vector gradiente

El vector gradiente

Definición

El gradiente de $f : \mathbb{R}^n \to \mathbb{R}$ en $\mathbf{x}$ es el vector de todas sus derivadas parciales:

$$\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n} \right)$$

- Para $n = 2$: $\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$.
- El gradiente es un **campo vectorial**: a cada punto del dominio le asocia un vector.
- Propiedades: linealidad $\nabla(af + bg) = a\nabla f + b\nabla g$; regla del producto $\nabla(fg) = g\nabla f + f\nabla g$.

Ejemplo

$f(x, y) = x^2 + 2y^2 \;\Rightarrow\; \nabla f(x, y) = (2x,\, 4y)$, y en $(1, 1)$: $\nabla f(1, 1) = (2, 4)$.

Gradiente en Python (simbólico y numérico)

```python
import sympy as sp

x, y = sp.symbols('x y')
f = x**2 + 2*y**2
grad_f = [sp.diff(f, v) for v in (x, y)]
print(grad_f)                          # [2*x, 4*y]

punto = {x: 1, y: 1}
grad_en_punto = [g.subs(punto) for g in grad_f]
print(grad_en_punto)                   # [2, 4]
```

Idea clave

El gradiente generaliza la derivada a varias variables: **apunta en la dirección en la que** f **crece más rápido** (lo demostramos en breve) y su magnitud indica **qué tan rápido** crece en esa dirección.

Curvas de nivel

Curvas de nivel (conjuntos de nivel)

Definición

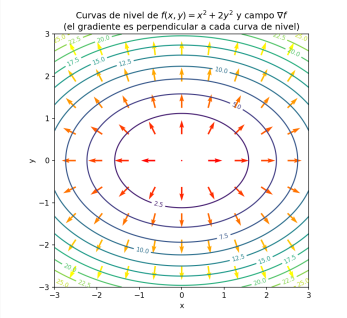
La curva (o conjunto) de nivel c de $f : \mathbb{R}^2 \to \mathbb{R}$ es

$$\{(x, y) \in \mathbb{R}^2 : f(x, y) = c\}$$

es decir, todos los puntos donde f toma el **mismo valor**.

- Son el análogo matemático de las curvas de nivel de un mapa topográfico (líneas de igual altitud).
- Al variar c se obtiene una familia de curvas que *rebanan* la superficie horizontalmente.
- Curvas de nivel muy juntas $\Rightarrow$ la función cambia muy rápido (pendiente pronunciada); muy separadas $\Rightarrow$ cambia lento.

Curvas de nivel y el gradiente



Teorema (perpendicularidad)

En todo punto, ∇f es **perpendicular** a la curva de nivel que pasa por ese punto, y apunta hacia donde f **crece**.

Curvas de nivel + gradiente en Python

```python
import numpy as np
import matplotlib.pyplot as plt

f    = lambda x, y: x**2 + 2*y**2
grad = lambda x, y: (2*x, 4*y)

x = np.linspace(-3, 3, 300); y = np.linspace(-3, 3, 300)
X, Y = np.meshgrid(x, y)
plt.contour(X, Y, f(X, Y), levels=12, cmap="viridis")   # curvas de nivel

xg, yg = np.meshgrid(np.linspace(-2.5,2.5,9), np.linspace(-2.5,2.5,9))
U, V = grad(xg, yg)
plt.quiver(xg, yg, U, V)                                  # campo gradiente
plt.gca().set_aspect("equal")
plt.show()
```

Script completo: `scripts/03_curvas_nivel_gradiente.py`

Derivada direccional y máximo crecimiento

Derivada direccional: definición

Definición

Dado un vector unitario $\mathbf{u}$ ($\|\mathbf{u}\| = 1$), la derivada direccional de f en $\mathbf{x}_0$ en la dirección $\mathbf{u}$ es

$$D_{\mathbf{u}} f(\mathbf{x}_0) = \lim_{t \to 0} \frac{f(\mathbf{x}_0 + t\mathbf{u}) - f(\mathbf{x}_0)}{t}$$

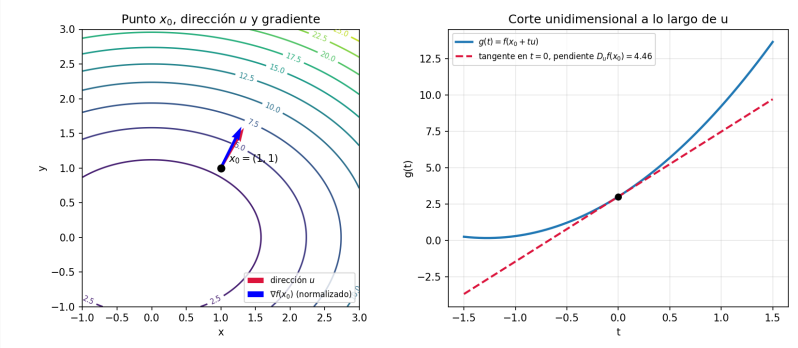
- Mide qué tan rápido cambia f si nos movemos desde $\mathbf{x}_0$ en la dirección $\mathbf{u}$.
- Caso particular: si $\mathbf{u} = (1, 0)$, $D_{\mathbf{u}} f = \partial f / \partial x$.

Fórmula práctica (si f es diferenciable)

$$D_{\mathbf{u}} f(\mathbf{x}_0) = \nabla f(\mathbf{x}_0) \cdot \mathbf{u}$$

¡No hace falta calcular un límite nuevo cada vez! Basta con el gradiente.

Derivada direccional: visualización



Izquierda: punto x_0, dirección u y $\nabla f(x_0)$ sobre las curvas de nivel. Derecha: corte $g(t) = f(x_0 + tu)$; la pendiente de su tangente en $t = 0$ es $D_u f(x_0)$.

El gradiente como dirección de máximo crecimiento

Usando $D_{\mathbf{u}} f(\mathbf{x}_0) = \nabla f(\mathbf{x}_0) \cdot \mathbf{u} = \|\nabla f(\mathbf{x}_0)\| \, \|\mathbf{u}\| \cos\theta$, con θ el ángulo entre $\nabla f(\mathbf{x}_0)$ y $\mathbf{u}$, y como $\|\mathbf{u}\| = 1$:

$$D_{\mathbf{u}} f(\mathbf{x}_0) = \|\nabla f(\mathbf{x}_0)\| \cos\theta$$

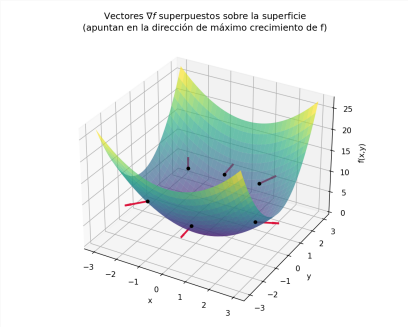
Consecuencia

$D_{\mathbf{u}} f(\mathbf{x}_0)$ es **máxima** cuando $\cos\theta = 1$, es decir, cuando $\mathbf{u}$ apunta **en la misma dirección que** $\nabla f(\mathbf{x}_0)$.

- Dirección de **máximo ascenso**: $\mathbf{u} = \nabla f(\mathbf{x}_0) / \|\nabla f(\mathbf{x}_0)\|$.
- Dirección de **máximo descenso**: $\mathbf{u} = -\nabla f(\mathbf{x}_0) / \|\nabla f(\mathbf{x}_0)\|$.

Este hecho es la base del **método de descenso de gradiente**, que veremos en una próxima clase: para minimizar, uno se mueve repetidamente en la dirección $-\nabla f(\mathbf{x})$.

Gradientes superpuestos sobre la superficie



En cada punto de la superficie, el vector gradiente (proyectado en el plano xy) apunta hacia donde f aumenta más rápido: aquí, siempre *cuesta arriba* alejándose del mínimo en el origen.

Monotonía

Monotonía

Una variable

f es **creciente** en un intervalo si $f'(x) > 0$ allí, y **decreciente** si $f'(x) < 0$. Si $f'(x) = 0$ en un intervalo, f es constante allí.

Varias variables (a lo largo de una dirección)

f es creciente en $\mathbf{x}_0$ en la dirección $\mathbf{u}$ si $D_{\mathbf{u}} f(\mathbf{x}_0) > 0$, y decreciente si $D_{\mathbf{u}} f(\mathbf{x}_0) < 0$.

Ejemplo

Para $f(x, y) = x^2 + 2y^2$ en $\mathbf{x}_0 = (1, 1)$, moverse en la dirección $\mathbf{u} = \nabla f(\mathbf{x}_0)/\|\nabla f(\mathbf{x}_0)\|$ hace crecer f lo más rápido posible ($D_{\mathbf{u}} f(\mathbf{x}_0) = \|\nabla f(\mathbf{x}_0)\| > 0$); moverse en $-\mathbf{u}$ la hace decrecer lo más rápido posible.

Puntos críticos

Puntos críticos: condición necesaria

Definición

$\mathbf{x}^*$ es un **punto crítico** (o estacionario) de f si

$$\nabla f(\mathbf{x}^*) = \mathbf{0}$$

(en una variable: $f'(x^*) = 0$).

Teorema (condición necesaria de primer orden)

Si $\mathbf{x}^*$ es un mínimo o máximo **local** de f (diferenciable, sin restricciones), entonces $\mathbf{x}^*$ es un punto crítico.

- **No todo punto crítico es un mínimo o máximo**: también puede ser un **punto de silla** (más adelante).
- Buscar óptimos $\Rightarrow$ primero encontrar dónde $\nabla f(\mathbf{x}) = \mathbf{0}$, y luego clasificar cada punto crítico.

Ejemplo

$f(x, y) = x^2 + 2y^2$: $\nabla f = (2x, 4y) = (0, 0) \Rightarrow (x, y) = (0, 0)$ es el único punto crítico.

La matriz Hessiana

La matriz Hessiana

Definición

La Hessiana de $f : \mathbb{R}^n \to \mathbb{R}$ (dos veces diferenciable) es la matriz de segundas derivadas parciales:

$$\nabla^2 f(\mathbf{x}) = \begin{pmatrix} \dfrac{\partial^2 f}{\partial x_1^2} & \cdots & \dfrac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \dfrac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$

Bajo condiciones usuales de continuidad, $\nabla^2 f$ es **simétrica**.

Ejemplo

$f(x, y) = x^2 + 2y^2 \;\Rightarrow\; \nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$ (constante en todo el dominio, por ser f cuadrática).

Matrices definidas positivas / negativas

Definición

Una matriz simétrica H es:

- **Definida positiva** si $\mathbf{d}^\top H\mathbf{d} > 0$ para todo $\mathbf{d} \neq \mathbf{0} \Leftrightarrow$ todos sus autovalores > 0.
- **Definida negativa** si $\mathbf{d}^\top H\mathbf{d} < 0$ para todo $\mathbf{d} \neq \mathbf{0} \Leftrightarrow$ todos sus autovalores < 0.
- **Indefinida** si toma ambos signos $\Leftrightarrow$ tiene autovalores de ambos signos.

Criterio práctico en 2×2

Para $H = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$, con $\det H = ac - b^2$:

- $\det H > 0$ y $a > 0 \Rightarrow$ definida positiva.
- $\det H > 0$ y $a < 0 \Rightarrow$ definida negativa.
- $\det H < 0 \Rightarrow$ indefinida (punto de silla).

Clasificación de puntos críticos

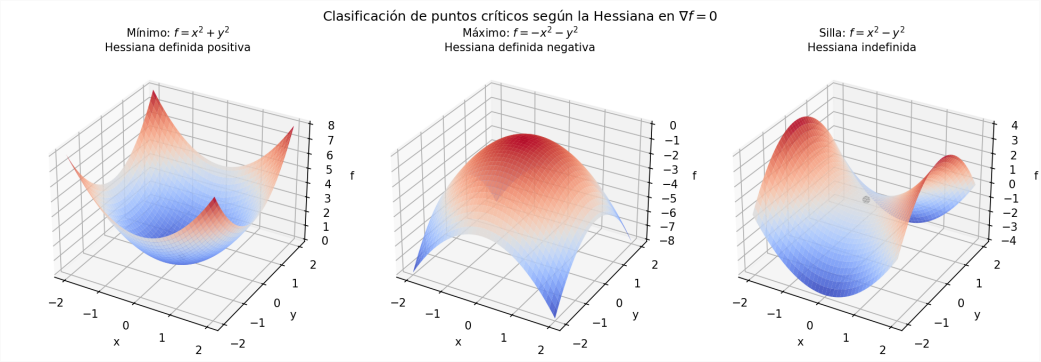
Criterio de la segunda derivada

Sea $\mathbf{x}^*$ un punto crítico ($\nabla f(\mathbf{x}^*) = \mathbf{0}$):

- $\nabla^2 f(\mathbf{x}^*)$ definida positiva $\Rightarrow$ **mínimo local** estricto.
- $\nabla^2 f(\mathbf{x}^*)$ definida negativa $\Rightarrow$ **máximo local** estricto.
- $\nabla^2 f(\mathbf{x}^*)$ indefinida $\Rightarrow$ **punto de silla** (ni máximo ni mínimo).
- $\nabla^2 f(\mathbf{x}^*)$ semidefinida (con autovalor 0) $\Rightarrow$ el criterio **no decide**; se requiere más análisis.

(En una variable, esto es el conocido criterio $f''(x^*) > 0 \Rightarrow$ mínimo, $f''(x^*) < 0 \Rightarrow$ máximo.)

Los tres casos, visualizados



Clasificar puntos críticos en Python

```python
import numpy as np

H = np.array([[2, 0],
              [0, -4]])                  # Hessiana en el punto critico

eigvals = np.linalg.eigvalsh(H)          # autovalores (H simetrica)
print(eigvals)                           # [-4.  2.]

if np.all(eigvals > 0):
    print("minimo local")
elif np.all(eigvals < 0):
    print("maximo local")
elif np.any(eigvals > 0) and np.any(eigvals < 0):
    print("punto de silla")
else:
    print("caso no decidido por el criterio (autovalor 0)")
```

Serie de Taylor

Serie de Taylor en una variable

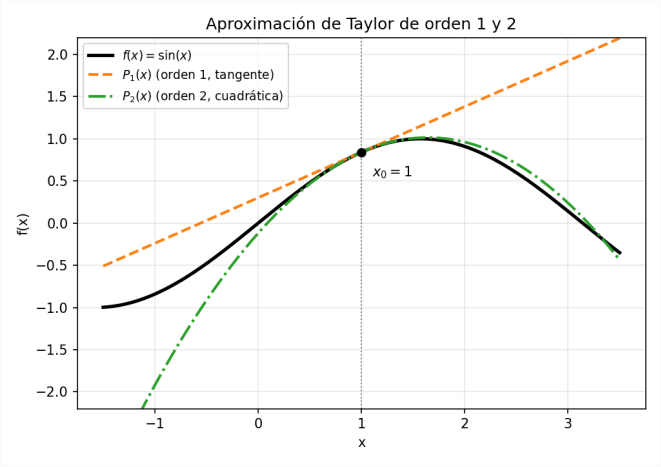
Idea

Aproximar f cerca de x_0 mediante un polinomio construido con sus derivadas en x_0:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2!}(x - x_0)^2 + \dots$$

- **Orden 1** (lineal): $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$ — es la recta tangente.
- **Orden 2** (cuadrático): $f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2$ — captura también la **curvatura**.
- Cuanto mayor el orden, mejor la aproximación **cerca** de x_0 (pero puede empeorar lejos de x_0).

Serie de Taylor: visualización



$f(x) = \sin(x)$ cerca de $x_0 = 1$: la aproximación cuadrática P_2 sigue a f mucho más de cerca que la lineal P_1.

Serie de Taylor multivariable

Para $f : \mathbb{R}^n \to \mathbb{R}$, alrededor de $\mathbf{x}_0$ y con incremento $\mathbf{d}$:

$$f(\mathbf{x}_0 + \mathbf{d}) \approx \underbrace{f(\mathbf{x}_0)}_{\text{orden } 0} + \underbrace{\nabla f(\mathbf{x}_0)^\top \mathbf{d}}_{\text{orden } 1} + \underbrace{\tfrac{1}{2}\, \mathbf{d}^\top \nabla^2 f(\mathbf{x}_0)\, \mathbf{d}}_{\text{orden } 2}$$

Por qué importa en optimización

La aproximación de **orden 2** es un **modelo cuadrático local** de f: los métodos de optimización tipo **Newton** minimizan ese modelo cuadrático en cada paso en lugar de f directamente (lo veremos en una próxima clase).

- Si $\mathbf{x}_0$ es un punto crítico ($\nabla f(\mathbf{x}_0) = \mathbf{0}$), el modelo se reduce a $f(\mathbf{x}_0) + \frac{1}{2}\mathbf{d}^\top \nabla^2 f(\mathbf{x}_0)\mathbf{d}$ — por eso la Hessiana clasifica el punto crítico.

Taylor en Python

```python
import numpy as np

f       = np.sin
fprime  = np.cos
fsecond = lambda x: -np.sin(x)

x0 = 1.0
x  = np.linspace(x0 - 2, x0 + 2, 200)

P1 = f(x0) + fprime(x0) * (x - x0)                    # orden 1
P2 = P1 + 0.5 * fsecond(x0) * (x - x0)**2             # orden 2
```

Script completo: `scripts/07_taylor_series.py`

Convexidad

Conjuntos y funciones convexas

Conjunto convexo

$\Omega \subseteq \mathbb{R}^n$ es convexo si el segmento que une dos puntos cualesquiera de Ω está contenido en Ω:

$$\mathbf{x}_1, \mathbf{x}_2 \in \Omega,\ t \in [0,1] \ \Rightarrow\ t\mathbf{x}_1 + (1-t)\mathbf{x}_2 \in \Omega$$

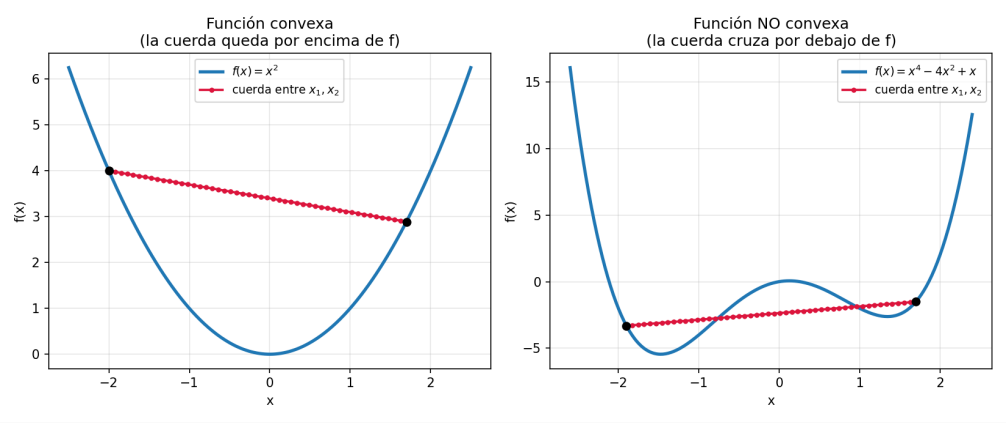
Función convexa

$f : \Omega \to \mathbb{R}$ (con Ω convexo) es convexa si

$$f\big(t\mathbf{x}_1 + (1-t)\mathbf{x}_2\big) \ \leq\ tf(\mathbf{x}_1) + (1-t)f(\mathbf{x}_2) \qquad \forall\, \mathbf{x}_1, \mathbf{x}_2 \in \Omega,\ t \in [0,1]$$

Interpretación: la **cuerda** entre dos puntos de la gráfica queda **siempre por encima** (o sobre) la función. Si la desigualdad es estricta para $\mathbf{x}_1 \neq \mathbf{x}_2$, $t \in (0,1)$, es **estrictamente convexa**.

Convexidad: visualización



Caracterizaciones de convexidad

Si f es diferenciable, son equivalentes a la definición anterior:

Caracterización de primer orden

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) \qquad \forall\, \mathbf{x}, \mathbf{y} \in \Omega$$

(la función siempre queda **por encima** de su plano tangente).

Caracterización de segundo orden

Si f es dos veces diferenciable, f es convexa $\Leftrightarrow$ $\nabla^2 f(\mathbf{x})$ es **semidefinida positiva** para todo $\mathbf{x} \in \Omega$ (y *definida* positiva $\Rightarrow$ estrictamente convexa).

En una variable: f convexa $\Leftrightarrow$ $f''(x) \geq 0$ en todo el dominio.

¿Por qué importa la convexidad?

Teorema central de la optimización convexa

Si f es convexa (sobre Ω convexo), entonces **todo mínimo local es también un mínimo global**.

- Elimina el problema de quedar *atrapado* en un mínimo local subóptimo.
- Es la razón por la que la **optimización convexa** es una de las áreas mejor entendidas y más usadas (regresión, SVM, control, finanzas, muchos problemas de ingeniería).
- Muchos algoritmos (descenso de gradiente, Newton) tienen **garantías teóricas fuertes** solo cuando f es convexa.

Ejemplo

$f(x, y) = x^2 + 2y^2$ tiene $\nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}$, definida positiva en todo punto $\Rightarrow$ f es (estrictamente) convexa $\Rightarrow$ su único punto crítico $(0, 0)$ es el mínimo global.

Verificar convexidad en Python (numéricamente)

```python
import numpy as np

def es_semidef_positiva(H, tol=1e-9):
    autovalores = np.linalg.eigvalsh(H)
    return np.all(autovalores >= -tol)

H = np.array([[2, 0],
              [0, 4]])
print(es_semidef_positiva(H))      # True  -> f convexa en ese punto
```

Script completo: `scripts/08_convexidad.py`

Máximos y mínimos: locales vs. globales

Mínimos y máximos: local vs. global

Definiciones

$\mathbf{x}^* \in \Omega$ es:

- **mínimo local** si $f(\mathbf{x}^*) \leq f(\mathbf{x})$ para todo $\mathbf{x}$ en un entorno de $\mathbf{x}^*$;
- **mínimo global** si $f(\mathbf{x}^*) \leq f(\mathbf{x})$ para **todo** $\mathbf{x} \in \Omega$.

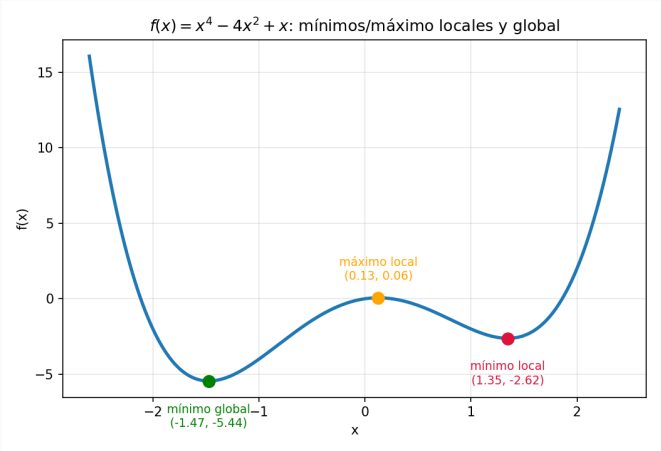
(análogo para máximos, invirtiendo la desigualdad).

Teorema de Weierstrass (existencia)

Si Ω es **compacto** (cerrado y acotado) y f es **continua**, entonces f alcanza un mínimo global y un máximo global en Ω.

Sin compacidad no hay garantía: por ejemplo $f(x) = x$ en $\mathbb{R}$ no tiene mínimo ni máximo.

Ejemplo: varios mínimos locales



$f(x) = x^4 - 4x^2 + x$ tiene dos mínimos locales, pero solo uno es el mínimo **global** (el de menor valor de f); en medio hay un máximo local. Esta función **no es convexa** (por eso puede tener más de un mínimo local).

Ubicar puntos críticos en Python

```python
import numpy as np

# f(x) = x^4 - 4x^2 + x  ->  f'(x) = 4x^3 - 8x + 1
coef_fprime = [4, 0, -8, 1]
raices = np.roots(coef_fprime)
raices = np.sort(raices[np.isreal(raices)].real)

fsecond = lambda x: 12*x**2 - 8
for r in raices:
    tipo = "minimo" if fsecond(r) > 0 else "maximo"
    print(f"x*={r:.3f}  tipo={tipo}  f(x*)={r**4-4*r**2+r:.3f}")
```

Script completo: `scripts/09_min_local_global.py`

Optimización sin restricciones: síntesis

Condiciones de optimalidad sin restricciones

Para $\min_{\mathbf{x}\in\mathbb{R}^n} f(\mathbf{x})$, con f dos veces diferenciable

- **Condición necesaria de primer orden:** si $\mathbf{x}^*$ es mínimo local, $\nabla f(\mathbf{x}^*) = \mathbf{0}$.
- **Condición suficiente de segundo orden:** si $\nabla f(\mathbf{x}^*) = \mathbf{0}$ y $\nabla^2 f(\mathbf{x}^*)$ es definida positiva, entonces $\mathbf{x}^*$ es mínimo local estricto.
- Si además f es **convexa**, todo mínimo local (equivalentemente, todo punto con $\nabla f = \mathbf{0}$) es **mínimo global**.

Receta general (a mano)

1) Calcular ∇f y resolver $\nabla f(\mathbf{x}) = \mathbf{0}$ (puntos críticos). 2) Calcular $\nabla^2 f$ en cada punto crítico y clasificarlo. 3) Si aplica, verificar convexidad para garantizar optimalidad global.

Restricciones de igualdad: multiplicadores de Lagrange

Optimización con restricciones de igualdad

$$\min_{\mathbf{x}\in\mathbb{R}^n} f(\mathbf{x}) \quad \text{s.a.} \quad h(\mathbf{x}) = 0$$

- $h : \mathbb{R}^n \to \mathbb{R}$ define una **superficie/curva de restricción**: solo podemos buscar el óptimo **sobre** ella.
- Ejemplo: maximizar el área de un rectángulo de lados x, y con perímetro fijo $2x + 2y = P$.

Idea geométrica

En el óptimo restringido $\mathbf{x}^*$, **no puede mejorar** moverse a lo largo de la restricción; esto ocurre exactamente cuando $\nabla f(\mathbf{x}^*)$ es **paralelo** a $\nabla h(\mathbf{x}^*)$ (perpendicular ambos a la curva de restricción en $\mathbf{x}^*$).

El método de los multiplicadores de Lagrange

Función Lagrangiana

$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) - \lambda\, h(\mathbf{x})$$

Condiciones necesarias de primer orden

$$\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{0} \implies \nabla f(\mathbf{x}^*) = \lambda \nabla h(\mathbf{x}^*), \qquad h(\mathbf{x}^*) = 0$$

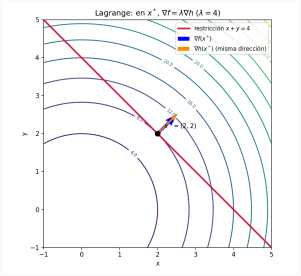
- λ (**multiplicador de Lagrange**) mide cuánto cambiaría el óptimo de f si se relajara ligeramente la restricción.
- Da un sistema de $n+1$ ecuaciones ($\nabla f = \lambda \nabla h$ y $h = 0$) con $n+1$ incógnitas ($\mathbf{x}^*, \lambda$).

Ejemplo resuelto

$$\min_{x,y} \; f(x,y) = x^2 + y^2 \qquad \text{s.a.} \qquad h(x,y) = x + y - 4 = 0$$

$$\nabla f = (2x, 2y), \quad \nabla h = (1,1) \;\Rightarrow\; 2x = \lambda,\; 2y = \lambda \;\Rightarrow\; x = y$$

Con $x + y = 4$: $x = y = 2$, $\lambda = 4$.



Lagrange en Python (con `sympy`)

```python
import sympy as sp

x, y, lam = sp.symbols('x y lambda')
f = x**2 + y**2
h = x + y - 4

L = f - lam*h
ecuaciones = [sp.diff(L, v) for v in (x, y, lam)]
solucion = sp.solve(ecuaciones, (x, y, lam))
print(solucion)      # {x: 2, y: 2, lambda: 4}
```

Script completo (versión gráfica): `scripts/10_lagrange_igualdad.py`

Restricciones de desigualdad: introducción a KKT

Optimización con restricciones de desigualdad

$$\min_{\mathbf{x}\in\mathbb{R}^n} f(\mathbf{x}) \quad \text{s.a.} \quad g(\mathbf{x}) \leq 0$$

- El conjunto factible es una **región** (no solo una curva), p. ej. $g(\mathbf{x}) \leq 0$ recorta un semiespacio, un semiplano curvo, etc.
- En el óptimo, la restricción puede estar:
  - **activa**: $g(\mathbf{x}^*) = 0$ (el óptimo está sobre la frontera);
  - **inactiva**: $g(\mathbf{x}^*) < 0$ (el óptimo está en el interior — la restricción *no estorba*, como si no existiera).

Idea clave

Si el mínimo *sin* restricciones ya es factible, la restricción de desigualdad es inactiva y el problema se resuelve como sin restricciones. Si no, el óptimo se *empuja* hasta la frontera.

Condiciones de Karush–Kuhn–Tucker (KKT)

Para mín $f(\mathbf{x})$ s.a. $g(\mathbf{x}) \leq 0$ (caso con una sola restricción, por simplicidad), $\mathbf{x}^*$ óptimo requiere que exista $\mu^* \geq 0$ con:

Condiciones KKT

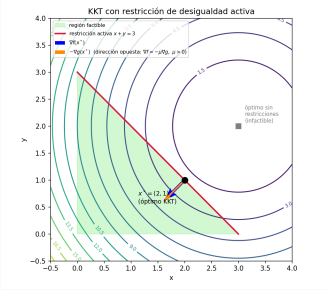
1. **Estacionariedad:** $\nabla f(\mathbf{x}^*) + \mu^* \nabla g(\mathbf{x}^*) = \mathbf{0}$
2. **Factibilidad primal:** $g(\mathbf{x}^*) \leq 0$
3. **Factibilidad dual:** $\mu^* \geq 0$
4. **Holgura complementaria:** $\mu^* \, g(\mathbf{x}^*) = 0$

La condición 4 formaliza la idea de activa/inactiva: si $g(\mathbf{x}^*) < 0$ (inactiva) entonces $\mu^* = 0$ forzosamente (no *empuja*); si $\mu^* > 0$ entonces $g(\mathbf{x}^*) = 0$ (activa). Estas condiciones son la generalización natural de Lagrange a desigualdades; las estudiaremos en profundidad más adelante.

Ejemplo resuelto

$$\min_{x,y} \; f(x,y) = (x-3)^2 + (y-2)^2 \quad \text{s.a.} \quad x + y \le 3,\; x \ge 0,\; y \ge 0$$

El mínimo sin restricciones, $(3, 2)$, es **infactible** $(3 + 2 = 5 > 3) \Rightarrow$ la restricción $x + y \le 3$ está **activa**. Resolviendo KKT sobre esa restricción: $\mathbf{x}^* = (2, 1)$, $\mu^* = 2 \ge 0$ (válido).



KKT en Python (con `scipy.optimize`)

```python
from scipy.optimize import minimize

f = lambda v: (v[0]-3)**2 + (v[1]-2)**2
restricciones = [
    {"type": "ineq", "fun": lambda v: 3 - v[0] - v[1]},  # x+y<=3
    {"type": "ineq", "fun": lambda v: v[0]},              # x>=0
    {"type": "ineq", "fun": lambda v: v[1]},              # y>=0
]
sol = minimize(f, x0=[0, 0], constraints=restricciones)
print(sol.x)      # ~ [2. 1.]
```

Script completo (versión gráfica): `scripts/11_kkt_desigualdad.py`

Síntesis de la clase

Mapa de conceptos

Concepto	**Para qué sirve en optimización**
Derivada / parcial	Medir razón de cambio
Gradiente ∇f	Generaliza la derivada; dirección de máx. cambio
Curvas de nivel	Visualizar f; $\nabla f \perp$ curva de nivel
Derivada direccional	Cambio de f en cualquier dirección
Puntos críticos	Candidatos a óptimo ($\nabla f = \mathbf{0}$)
Hessiana	Clasifica puntos críticos (mín / máx / silla)
Serie de Taylor	Modelo cuadrático local (base de métodos tipo Newton)
Convexidad	Garantiza que mínimo local = mínimo global
Lagrange	Óptimo con restricciones de igualdad
KKT	Óptimo con restricciones de igualdad y desigualdad

Fórmulas clave (resumen)

Gradiente:	$\nabla f(\mathbf{x}) = (\partial f/\partial x_1, \ldots, \partial f/\partial x_n)$
Derivada direccional:	$D_{\mathbf{u}} f(\mathbf{x}) = \nabla f(\mathbf{x}) \cdot \mathbf{u}$
Taylor orden 2:	$f(\mathbf{x}_0 + \mathbf{d}) \approx f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^\top \mathbf{d} + \frac{1}{2}\mathbf{d}^\top \nabla^2 f(\mathbf{x}_0)\mathbf{d}$
Convexidad (1er orden):	$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x})$
Punto crítico:	$\nabla f(\mathbf{x}^*) = \mathbf{0}$
Lagrange:	$\nabla f(\mathbf{x}^*) = \lambda \nabla h(\mathbf{x}^*),\ h(\mathbf{x}^*) = 0$
KKT (estacionariedad):	$\nabla f(\mathbf{x}^*) + \mu^* \nabla g(\mathbf{x}^*) = \mathbf{0},\ \mu^* \geq 0,\ \mu^* g(\mathbf{x}^*) = 0$

¿Qué sigue en el curso?

Con este lenguaje matemático ya podemos estudiar **métodos** concretos de optimización:

- **Clase 2:** Métodos de descenso de gradiente (tasa de aprendizaje, convergencia, casos convexos vs. no convexos).
- **Clase 3:** Método de Newton y cuasi-Newton (uso del modelo cuadrático de Taylor).
- **Clase 4:** Programación lineal (simplex, dualidad).
- **Clase 5:** Programación no lineal con restricciones, KKT en profundidad, penalización y barrera.
- **Clases posteriores:** optimización estocástica, metaheurísticas, aplicaciones.

Ejercicios propuestos

Ejercicios para la próxima clase

1. Calcule ∇f y los puntos críticos de $f(x, y) = x^3 - 3xy + y^3$; clasifíquelos con la Hessiana.
2. Muestre, usando la definición de la cuerda, que $f(x) = e^x$ es convexa. Verifíquelo también con $f''(x)$.
3. Encuentre la dirección de máximo ascenso de $f(x, y) = \ln(x^2 + y^2 + 1)$ en el punto $(1, 2)$.
4. Resuelva con multiplicadores de Lagrange: maximizar $f(x, y) = xy$ sujeto a $x + y = 10$. Interprete λ.
5. Plantee las condiciones KKT para $\min (x - 1)^2 + (y - 1)^2$ s.a. $x^2 + y^2 \leq 1$, y resuélvalas a mano.
6. **Código:** modifique `scripts/09_min_local_global.py` para $f(x) = x^4 - 5x^2 + 4$ y reporte todos los mínimos/máximos locales y cuál es global.
7. **Código:** modifique `scripts/03_curvas_nivel_gradiente.py` para $f(x, y) = \sin(x)\cos(y)$ y comente qué observa en el campo gradiente cerca de los puntos de silla.

Referencias

Referencias

- S. Boyd, L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. (Disponible gratis en línea.)
- J. Nocedal, S. Wright. *Numerical Optimization*, 2.ª ed. Springer, 2006.
- D. Bertsekas. *Nonlinear Programming*, 3.ª ed. Athena Scientific, 2016.
- J. Stewart. *Cálculo de varias variables: trascendentes tempranas*. Cengage Learning.
- T. Apostol. *Calculus*, Vol. 2. Wiley.
- Documentación de `NumPy`, `SciPy` y `Matplotlib`: `https://numpy.org`, `https://scipy.org`, `https://matplotlib.org`.

¿Preguntas?